

Функция Эйлера

Определение

Функция Эйлера $\phi(n)$ (иногда обозначаемая $\varphi(n)$ или $\text{phi}(n)$) — это количество чисел от 1 до n , взаимно простых с n . Иными словами, это количество таких чисел в отрезке $[1; n]$, **наибольший общий делитель** которых с n равен единице.

Несколько первых значений этой функции ([A000010](#) в энциклопедии OEIS):

$$\begin{aligned}\phi(1) &= 1, \\ \phi(2) &= 1, \\ \phi(3) &= 2, \\ \phi(4) &= 2, \\ \phi(5) &= 4.\end{aligned}$$

Свойства

Три следующих простых свойства функции Эйлера — достаточны, чтобы научиться вычислять её для любых чисел:

- Если p — простое число, то $\phi(p) = p - 1$.
(Это очевидно, т.к. любое число, кроме самого p , взаимно просто с ним.)
- Если p — простое, a — натуральное число, то $\phi(p^a) = p^a - p^{a-1}$.
(Поскольку с числом p^a не взаимно просты только числа вида pk ($k \in \mathcal{N}$), которых $p^a/p = p^{a-1}$ штук.)
- Если a и b взаимно простые, то $\phi(ab) = \phi(a)\phi(b)$ ("мультипликативность" функции Эйлера).
(Этот факт следует из [китайской теоремы об остатках](#). Рассмотрим произвольное число $z \leq ab$. Обозначим через x и y остатки от деления z на a и b соответственно. Тогда z взаимно просто с ab тогда и только тогда, когда z взаимно просто с a и с b по отдельности, или, что то же самое, x взаимно просто с a и y взаимно просто с b . Применяя китайскую теорему об остатках, получаем, что любой паре чисел x и y ($x \leq a, y \leq b$) взаимно однозначно соответствует число z ($z \leq ab$), что и завершает доказательство.)

Отсюда можно получить функцию Эйлера для любого n через его **факторизацию** (разложение n на простые сомножители):

если

$$n = p_1^{a_1} \cdot p_2^{a_2} \cdot \dots \cdot p_k^{a_k}$$

(где все p_i — простые), то

$$\begin{aligned}\phi(n) &= \phi(p_1^{a_1}) \cdot \phi(p_2^{a_2}) \cdot \dots \cdot \phi(p_k^{a_k}) = \\ &= (p_1^{a_1} - p_1^{a_1-1}) \cdot (p_2^{a_2} - p_2^{a_2-1}) \cdot \dots \cdot (p_k^{a_k} - p_k^{a_k-1}) = \\ &= n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdot \dots \cdot \left(1 - \frac{1}{p_k}\right).\end{aligned}$$

Реализация

Простейший код, вычисляющий функцию Эйлера, факторизуя число элементарным методом за $O(\sqrt{n})$:

```
int phi (int n) {
    int result = n;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
```

```
}
if (n > 1)
    result -= result / n;
return result;
}
```

Ключевое место для вычисление функции Эйлера — это нахождение **факторизации** числа n . Его можно осуществить за время, значительно меньшее $O(\sqrt{n})$: см. [Эффективные алгоритмы факторизации](#).

Приложения функции Эйлера

Самое известное и важное свойство функции Эйлера выражается в **теореме Эйлера**:

$$a^{\phi(m)} \equiv 1 \pmod{m},$$

где a и m взаимно просты.

В частном случае, когда m простое, теорема Эйлера превращается в так называемую **малую теорему Ферма**:

$$a^{m-1} \equiv 1 \pmod{m}$$

Теорема Эйлера достаточно часто встречается в практических приложениях, например, см. [Обратный элемент в поле по модулю](#).

Задачи в online judges

Список задач, в которых требуется посчитать функцию Эйлера, либо воспользоваться теоремой Эйлера, либо по значению функции Эйлера восстанавливать исходное число:

- [UVA #10179 "Irreducible Basic Fractions"](#) [сложность: низкая]
- [UVA #10299 "Relatives"](#) [сложность: низкая]
- [UVA #11327 "Enumerating Rational Numbers"](#) [сложность: средняя]
- [TIMUS #1673 "Допуск к экзамену"](#) [сложность: высокая]

Бинарное возведение в степень

Бинарное (двоичное) возведение в степень — это приём, позволяющий возводить любое число в n -ую степень за $O(\log n)$ умножений (вместо n умножений при обычном подходе).

Более того, описываемый здесь приём применим к любой **ассоциативной** операции, а не только к умножению чисел. Напомним, операция называется ассоциативной, если для любых a, b, c выполняется:

$$(a \cdot b) \cdot c = a \cdot (b \cdot c)$$

Наиболее очевидное обобщение — на остатки по некоторому модулю (очевидно, ассоциативность сохраняется). Следующим по "популярности" является обобщение на произведение матриц (его ассоциативность общеизвестна).

Алгоритм

Заметим, что для любого числа a и **чётного** числа n выполнимо очевидное тождество (следующее из ассоциативности операции умножения):

$$a^n = (a^{n/2})^2 = a^{n/2} \cdot a^{n/2}$$

Оно и является основным в методе бинарного возведения в степень. Действительно, для чётного n мы показали, как, потратив всего одну операцию умножения, можно свести задачу к вдвое меньшей степени.

Осталось понять, что делать, если степень n **нечётна**. Здесь мы поступаем очень просто: перейдём к степени $n - 1$, которая будет уже чётной:

$$a^n = a^{n-1} \cdot a$$

Итак, мы фактически нашли рекуррентную формулу: от степени n мы переходим, если она чётна, к $n/2$, а иначе — к $n - 1$. Понятно, что всего будет не более $2 \log n$ переходов, прежде чем мы придём к $n = 0$ (базе рекуррентной формулы). Таким образом, мы получили алгоритм, работающий за $O(\log n)$ умножений.

Реализация

Простейшая рекурсивная реализация:

```
int binpow (int a, int n) {
    if (n == 0)
        return 1;
    if (n % 2 == 1)
        return binpow (a, n-1) * a;
    else {
        int b = binpow (a, n/2);
        return b * b;
    }
}
```

Нерекурсивная реализация, оптимизированная (деления на 2 заменены битовыми операциями):

```
int binpow (int a, int n) {
    int res = 1;
    while (n)
        if (n & 1) {
            res *= a;
            --n;
        }
        else {
            a *= a;
            n >>= 1;
        }
    return res;
}
```

```
}
```

Эту реализацию можно ещё несколько упростить, заметив, что возведение a в квадрат осуществляется всегда, независимо от того, сработало условие нечётности n или нет:

```
int binpow (int a, int n) {
    int res = 1;
    while (n) {
        if (n & 1)
            res *= a;
        a *= a;
        n >>= 1;
    }
    return res;
}
```

Наконец, стоит отметить, что бинарное возведение в степень уже реализовано в языке Java, но только для класса длинной арифметики BigInteger (функция pow этого класса работает именно по алгоритму бинарного возведения).

Числа Фибоначчи

Определение

Последовательность Фибоначчи определяется следующим образом:

$$\begin{aligned} F_0 &= 0, \\ F_1 &= 1, \\ F_n &= F_{n-1} + F_{n-2}. \end{aligned}$$

Несколько первых её членов:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, , 55, 89, ...

История

Эти числа ввёл в 1202 г. Леонардо Фибоначчи (Leonardo Fibonacci) (также известный как Леонардо Пизанский (Leonardo Pisano)). Однако именно благодаря математику 19 века Люка (Lucas) название "числа Фибоначчи" стало общепотребительным.

Впрочем, индийские математики упоминали числа этой последовательности ещё раньше: Гопала (Gopala) до 1135 г., Хемачандра (Hemachandra) — в 1150 г.

Числа Фибоначчи в природе

Сам Фибоначчи упоминал эти числа в связи с такой задачей: "Человек посадил пару кроликов в загон, окруженный со всех сторон стеной. Сколько пар кроликов за год может произвести на свет эта пара, если известно, что каждый месяц, начиная со второго, каждая пара кроликов производит на свет одну пару?". Решением этой задачи и будут числа последовательности, называемой теперь в его честь. Впрочем, описанная Фибоначчи ситуация — больше игра разума, чем реальная природа.

Индийские математики Гопала и Хемачандра упоминали числа этой последовательности в связи с количеством ритмических рисунков, образующихся в результате чередования долгих и кратких слогов в стихах или сильных и слабых долей в музыке. Число таких рисунков, имеющих в целом *n* долей, равно *F_n*.

Числа Фибоначчи появляются и в работе Кеплера 1611 года, который размышлял о числах, встречающихся в природе (работа "О шестиугольных снежинках").

Интересен пример растения — тысячелистника, у которого число стеблей (а значит и цветков) всегда есть число Фибоначчи. Причина этого проста: будучи изначально с единственным стеблем, этот стебель затем делится на два, затем от главного стебля ответвляется ещё один, затем первые два стебля снова разветвляются, затем все стебли, кроме двух последних, разветвляются, и так далее. Таким образом, каждый стебель после своего появления "пропускает" одно разветвление, а затем начинает делиться на каждом уровне разветвлений, что и даёт в результате числа Фибоначчи.

Вообще говоря, у многих цветов (например, лилий) число лепестков является тем или иным числом Фибоначчи.

Также в ботанике известно явление "филлотаксиса". В качестве примера можно привести расположение семечек подсолнуха: если посмотреть сверху на их расположение, то можно увидеть одновременно две серии спиралей (как бы наложенных друг на друга): одни закручены по часовой стрелке, другие — против. Оказывается, что число этих спиралей примерно совпадает с двумя последовательными числами Фибоначчи: 34 и 55 или 89 и 144. Аналогичные факты верны и для некоторых других цветов, а также для сосновых шишек, брокколи, ананасов, и т.д.

Для многих растений (по некоторым данным, для 90% из них) верен и такой интересный факт. Рассмотрим какой-нибудь лист, и будем спускаться от него вниз до тех пор, пока не достигнем листа, расположенного на стебле точно так же (т.е. направленного точно в ту же сторону). Попутно будем считать все листья, попадавшие на нас (т.е. расположенные по высоте между стартовым листом и конечным), но расположенными по-другому. Нумеруя их, мы будем постепенно совершать витки вокруг стебля (поскольку листья расположены на стебле по спирали). В зависимости от того, совершать витки по часовой стрелке или против, будет получаться разное число витков. Но оказывается, что число витков, совершённых нами по часовой стрелке, число витков, совершённых против часовой стрелки, и число встреченных листьев образуют 3 последовательных числа Фибоначчи.

Впрочем, следует отметить, что есть и растения, для которых приведённые выше подсчёты дадут числа из совсем других последовательностей, поэтому нельзя сказать, что явление филлотаксиса является законом, — это

скорее занимательная тенденция.

Свойства

Числа Фибоначчи обладают множеством интересных математических свойств.

Вот лишь некоторые из них:

- Соотношение Кассини:

$$F_{n+1}F_{n-1} - F_n^2 = (-1)^n.$$

- Правило "сложения":

$$F_{n+k} = F_kF_{n+1} + F_{k-1}F_n.$$

- Из предыдущего равенства при $k = n$ вытекает:

$$F_{2n} = F_n(F_{n+1} + F_{n-1}).$$

- Из предыдущего равенства по индукции можно получить, что

$$F_{nk} \text{ всегда кратно } F_n.$$

- Верно и обратное к предыдущему утверждение:

$$\text{если } F_m \text{ кратно } F_n, \text{ то } m \text{ кратно } n.$$

- НОД-равенство:

$$\gcd(F_m, F_n) = F_{\gcd(m, n)}.$$

- По отношению к алгоритму Евклида числа Фибоначчи обладают тем замечательным свойством, что они являются наихудшими входными данными для этого алгоритма (см. "Теорема Ламе" в [Алгоритме Евклида](#)).

Фибоначчиева система счисления

Теорема Цекендорфа утверждает, что любое натуральное число n можно представить единственным образом в виде суммы чисел Фибоначчи:

$$N = F_{k_1} + F_{k_2} + \dots + F_{k_r},$$

где $k_1 \geq k_2 + 2, k_2 \geq k_3 + 2, \dots, k_r \geq 2$ (т.е. в записи нельзя использовать два соседних числа Фибоначчи).

Отсюда следует, что любое число можно однозначно записать в **фибоначчиевой системе счисления**, например:

$$\begin{aligned} 9 &= 8 + 1 = F_6 + F_1 = (10001)_F, \\ 6 &= 5 + 1 = F_5 + F_1 = (1001)_F, \\ 19 &= 13 + 5 + 1 = F_7 + F_5 + F_1 = (101001)_F, \end{aligned}$$

причём ни в каком числе не могут идти две единицы подряд.

Нетрудно получить и правило прибавления единицы к числу в фибоначчиевой системе счисления: если младшая цифра равна 0, то её заменяем на 1, а если равна 1 (т.е. в конце стоит 01), то 01 заменяем на 10. Затем "исправляем" запись, последовательно исправляя везде 011 на 100. В результате за линейное время будет получена запись нового числа.

Перевод числа в фибоначчиеву систему счисления осуществляется простым "жадным" алгоритмом: просто перебираем числа Фибоначчи от больших к меньшим и, если некоторое $F_k \leq n$, то F_k входит в запись числа n , и мы отнимаем F_k от n и продолжаем поиск.

Формула для n-го числа Фибоначчи

Формула через радикалы

Существует замечательная формула, называемая по имени французского математика Бине (Binet), хотя она была известна до него Муавру (Moivre):

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}.$$

Эту формулу легко доказать по индукции, однако вывести её можно с помощью понятия образующих функций или с помощью решения функционального уравнения.

Сразу можно заметить, что второе слагаемое всегда по модулю меньше 1, и более того, очень быстро убывает (экспоненциально). Отсюда следует, что значение первого слагаемого даёт "почти" значение F_n . Это можно записать в строгом виде:

$$F_n = \left\lfloor \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \right\rfloor,$$

где квадратные скобки обозначают округление до ближайшего целого.

Впрочем, для практического применения в вычислениях эти формулы мало подходят, потому что требуют очень высокой точности работы с дробными числами.

Матричная формула для чисел Фибоначчи

Нетрудно доказать матричное следующее равенство:

$$\begin{pmatrix} F_{n-2} & F_{n-1} \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} F_{n-1} & F_n \end{pmatrix}.$$

Но тогда, обозначая

$$P \equiv \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix},$$

получаем:

$$\begin{pmatrix} F_0 & F_1 \end{pmatrix} \cdot P^n = \begin{pmatrix} F_n & F_{n+1} \end{pmatrix}.$$

Таким образом, для нахождения n -го числа Фибоначчи надо возвести матрицу P в степень n .

Вспоминая, что возведение матрицы в n -ую степень можно осуществить за $O(\log n)$ (см. [Бинарное возведение в степень](#)), получается, что n -ое число Фибоначчи можно легко вычислить за $O(\log n)$ с использованием только целочисленной арифметики.

Периодичность последовательности Фибоначчи по модулю

Рассмотрим последовательность Фибоначчи F_i по некоторому модулю p . Докажем, что она является периодичной, и причём период начинается с $F_1 = 1$ (т.е. предпериод содержит только F_0).

Докажем это от противного. Рассмотрим $p^2 + 1$ пар чисел Фибоначчи, взятых по модулю p :

$$(F_1, F_2), (F_2, F_3), \dots, (F_{p^2+1}, F_{p^2+2}).$$

Поскольку по модулю p может быть только p^2 различных пар, то среди этой последовательности найдётся как минимум две одинаковые пары. Это уже означает, что последовательность периодична.

Выберем теперь среди всех таких одинаковых пар две одинаковые пары с наименьшими номерами. Пусть это пары с некоторыми номерами (F_a, F_{a+1}) и (F_b, F_{b+1}) . Докажем, что $a = 1$. Действительно, в противном случае для них найдутся предыдущие пары (F_{a-1}, F_a) и (F_{b-1}, F_b) , которые, по свойству чисел Фибоначчи, также будут равны друг другу. Однако это противоречит тому, что мы выбрали совпадающие пары с наименьшими номерами, что и требовалось доказать.

Литература

- Роналд Грэхэм, Дональд Кнут, Орен Паташник. **Конкретная математика** [1998]

Обратный элемент в кольце по модулю

Определение

Пусть задан некоторый натуральный модуль m , и рассмотрим кольцо, образуемое этим модулем (т.е. состоящее из чисел от 0 до $m - 1$). Тогда для некоторых элементов этого кольца можно найти **обратный элемент**.

Обратным к числу a по модулю m называется такое число b , что:

$$a \cdot b \equiv 1 \pmod{m},$$

и его нередко обозначают через a^{-1} .

Понятно, что для нуля обратного элемента не существует никогда; для остальных же элементов обратный может как существовать, так и нет. Утверждается, что обратный существует только для тех элементов a , которые **взаимно просты** с модулем m .

Рассмотрим ниже два способа нахождения обратного элемента, работающих при условии, что он существует.

В завершение, рассмотрим алгоритм, который позволяет найти обратные ко всех числам по некоторому модулю за линейное время.

Нахождение с помощью Расширенного алгоритма Евклида

Рассмотрим вспомогательное уравнение (относительно неизвестных x и y):

$$a \cdot x + m \cdot y = 1.$$

Это [линейное диофантово уравнение второго порядка](#). Как показано в соответствующей статье, из условия $\gcd(a, m) = 1$ следует, что это уравнение имеет решение, которое можно найти с помощью [Расширенного алгоритма Евклида](#) (отсюда же, кстати говоря, следует, что когда $\gcd(a, m) \neq 1$, решения, а потому и обратного элемента, не существует).

С другой стороны, если мы возьмём от обеих частей уравнения остаток по модулю m , то получим:

$$a \cdot x \equiv 1 \pmod{m}.$$

Таким образом, найденное x и будет являться обратным к a .

Реализация (с учётом того, что найденное x надо взять по модулю m , и x могло быть отрицательным):

```
int x, y;
int g = gcdex (a, m, x, y);
if (g != 1)
    cout << "no solution";
else {
    x = (x % m + m) % m;
    cout << x;
}
```

Асимптотика этого решения получается $O(\log m)$.

Нахождение с помощью Бинарного возведения в степень

Воспользуемся теоремой Эйлера:

$$a^{\phi(m)} \equiv 1 \pmod{m},$$

которая верна как раз для случая взаимно простых a и m .

Кстати говоря, в случае простого модуля m мы получаем ещё более простое утверждение — малую теорему Ферма:

$$a^{m-1} \equiv 1 \pmod{m}.$$

Умножим обе части каждого из уравнений на a^{-1} , получим:

- для любого модуля m :

$$a^{\phi(m)-1} \equiv a^{-1} \pmod{m},$$

- для простого модуля m :

$$a^{m-2} \equiv a^{-1} \pmod{m}.$$

Таким образом, мы получили формулы для непосредственного вычисления обратного. Для практического применения обычно используют эффективный [алгоритм бинарного возведения в степень](#), который в нашем случае позволит произвести возведение в степень за $O(\log m)$.

Этот метод представляется несколько проще описанного в предыдущем пункте, однако он требует знания значения функции Эйлера, что фактически требует факторизации модуля m , что иногда может оказаться весьма сложной задачей.

Если же факторизация числа известна, то тогда и этот метод также работает за асимптотику $O(\log m)$.

Нахождение всех простых по заданному модулю за линейное время

Пусть дан простой модуль m . Требуется для каждого числа в отрезке $[1; m - 1]$ найти обратное к нему.

Применяя описанные выше алгоритмы, мы получим лишь решения с асимптотикой $O(m \log m)$. Здесь же мы приведём простое решение с асимптотикой $O(m)$.

Решение это выглядит следующим образом. Обозначим через $r[i]$ искомое обратное к числу i по модулю m . Тогда для $i > 1$ верно тождество:

$$r[i] = - \left\lfloor \frac{m}{i} \right\rfloor \cdot r[m \bmod i]. \pmod{m}$$

Реализация этого удивительно лаконичного решения:

```
r[1] = 1;
for (int i=2; i<m; ++i)
    r[i] = (m - (m/i) * r[m%i] % m) % m;
```

Доказательство этого решения представляет из себя цепочку простых преобразований:

Распишем значение $m \bmod i$:

$$m \bmod i = m - \left\lfloor \frac{m}{i} \right\rfloor \cdot i,$$

откуда, беря обе части по модулю m , получаем:

$$m \bmod i = - \left\lfloor \frac{m}{i} \right\rfloor \cdot i. \pmod{m}$$

Умножая обе части на обратное к i и обратное к $(m \bmod i)$, получаем искомую формулу:

$$r[i] = - \left\lfloor \frac{m}{i} \right\rfloor \cdot r[m \bmod i], \pmod{m}$$

что и требовалось доказать.

